

Project: ISO JTC1/SC22/WG21: Programming Language C++
 Doc No: WG21 P0660R4
 Date: 2018-09-11
 Reply to: Nicolai Josuttis (nico@josuttis.de), Herb Sutter (hsutter@microsoft.com),
 Anthony Williams (anthony@justsoftwaresolutions.co.uk)
 Audience: SG1, LEWG, LWG
 Prev. Version: www.wg21.link/P0660R3

A Cooperatively Interruptible Joining Thread, Rev 4

New in R4

- Removed interruptible CV waiting members that don't take a predicate.
- Removed adding a new `cv_status` value `interrupted`.
- Added CV members for interruptible timed waits.
- Renamed CV members that wait interruptible.
- Several minor fixes (e.g. on `noexcept`) and full proposed wording.

Purpose

This is the proposed wording for a cooperatively interruptible joining thread.

For a full discussion of the motivation, see www.wg21.link/p0660r0 and www.wg21.link/p0660r1.

A default implementation exists at: github.com/josuttis/jthread. Note that the proposed functionality can be fully implemented on top of the existing C++ standard library.

Basis examples:

```
void testJThread()
{
    std::jthread t([] {
        while (...) {
            std::this_thread::throw_if_interrupted();
            ...
            // temporarily disable interrupts:
            auto origToken = std::this_thread::exchange_interrupt_token(std::interrupt_token{});
            ...
            std::this_thread::exchange_interrupt_token(origToken);
            ...
        }
    });
    ...
} // jthread destructor signals interrupt and therefore silently ends the started thread

void testInterruptibleCVWait()
{
    bool ready = false;
    std::mutex readyMutex;
    std::condition_variable readyCV;
    std::jthread t([&ready, &readyMutex, &readyCV] {
        while (...) {
            ...
            {
                std::unique_lock lg{readyMutex};
                readyCV.iwait_for(lg,
                                std::chrono::seconds(2),
                                [&ready] {
                                    return ready;
                                });
            }
            ...
        }
    });
    ...
} // jthread destructor signals interrupt and therefore unblocks the CV wait and silently ends the started thread
```

Feature Test Macro

This is a new feature so that it shall have the following feature macro:

```
__cpp_lib_jthread
```

Acknowledgements

Thanks to all who incredibly helped me to prepare this paper, such as all people in the C++ concurrency and library working group. Especially, we want to thank: Hans Boehm, Olivier Giroux, Pablo Halpern, Howard Hinnant, Alisdair Meredith, Gor Nishanov, Ville Voutilainen, and Jonathan Wakely.

Proposed Wording

All against N4659.

[*Editorial note:* This proposal uses the LaTeX macros of the draft standard. To adopt it please ask for the LaTeX source code of the proposed wording.]

30 Thread support library

[`thread`]

30.1 General

[`jthread.general`]

- ¹ The following subclauses describe components to create and manage threads (??), perform mutual exclusion, and communicate conditions and values between threads, as summarized in Table 1.

Table 1 — Thread support library summary

Subclause	Header(s)
30.2 Requirements	
30.3 Threads	<thread>
30.4 Interrupt Tokens	<interrupt_token>
30.5 Joining Threads	<jthread>
?? Mutual exclusion	<mutex> <shared_mutex>
30.6 Condition variables	<condition_variable>
?? Futures	<future>

30.2 Requirements

[`thread.req`]

...

30.3 Threads

[`thread.threads`]

...

30.3.2.2 `thread` constructors [`thread.thread.constr`]

`thread()` noexcept;

- ¹ *Effects:* Constructs a `thread` object that does not represent a thread of execution.

- ² *Ensures:* `get_id() == id()`.

template<class F, class... Args> explicit thread(F&& f, Args&&... args);

- ³ *Requires:* F and each T_i in Args shall satisfy the *Cpp17MoveConstructible* requirements. *INVOKE*(*DECAY_COPY*(std::forward<F>(f)), *DECAY_COPY*(std::forward<Args>(args))...) (??) shall be a valid expression.

- ⁴ *Remarks:* This constructor shall not participate in overload resolution if `remove_cvref_t<F>` is the same type as `std::thread`.

- ⁵ *Effects:* Constructs an object of type `thread`. The new thread of execution executes *INVOKE*(*DECAY_COPY*(std::forward<F>(f)), *DECAY_COPY*(std::forward<Args>(args))...) with the calls to *DECAY_COPY* being evaluated in the constructing thread. Any return value from this invocation is ignored. [Note: This implies that any exceptions not thrown from the invocation of the copy of f will be thrown in the constructing thread, not the new thread. — end note] If the invocation of *INVOKE*(*DECAY_COPY*(std::forward<F>(f)), *DECAY_COPY*(std::forward<Args>(args))...) terminates with an uncaught exception, `terminate` shall be called.

An uncaught **interrupted** exception in the started thread of execution will silently be ignored. [Note: Thus, an uncaught exception thrown by `this_thread::throw_if_interrupted()` will cause the started thread to end silently. — end note]

- ⁶ *Synchronization:* The completion of the invocation of the constructor synchronizes with the beginning of the invocation of the copy of f.

- ⁷ *Ensures:* `get_id() != id()`. `*this` represents the newly started thread.

- ⁸ *Throws:* `system_error` if unable to start the new thread.

- ⁹ *Error conditions:*

- (9.1) — `resource_unavailable_try_again` — the system lacked the necessary resources to create another thread, or the system-imposed limit on the number of threads in a process would be exceeded.

30.4 Interrupt Tokens

[`thread.interrupt_token`]

- ¹ 30.4 describes components that can be used to asynchronously signal an interrupt. The interrupt can only be signaled once.

30.4.1 Header `<interrupt_token>` synopsis

[`thread.interrupt_token.syn`]

```
namespace std {
    // 30.4.2 class interrupted
    class interrupted;

    // 30.4.3 class interrupt_token
    class interrupt_token;
}
```

30.4.2 Class `interrupted`

[`interrupted`]

```
namespace std {
    class interrupted {
    public:
        explicit interrupted() noexcept;
        explicit interrupted(const interrupted&) noexcept;
        interrupted& operator=(const interrupted&) noexcept;
        const char* what() const noexcept;
    };
}
```

- ¹ The class `interrupted` defines the type of objects thrown as exceptions by C++ standard library components, and certain expressions, to report a signaled interrupt. [*Note: This class is not derived from class `exception`. — end note*]

```
interrupted() noexcept;
```

- ² *Effects:* Constructs an object of class `interrupted`.

```
interrupted(const interrupted& rhs) noexcept;
interrupted& operator=(const interrupted& rhs) noexcept;
```

- ³ *Effects:* Copies an object of class `interrupted`.

- ⁴ *Ensures:* If `*this` and `rhs` both have dynamic type `interrupted` then the value of the expression `strcmp(what(), rhs.what())` shall equal 0.

```
const char* what() const noexcept;
```

- ⁵ *Returns:* An implementation-defined NTBS.

- ⁶ *Remarks:* The message may be a null-terminated multibyte string (`??`), suitable for conversion and display as a `wstring` (`??, ??`). The return value remains valid until the exception object from which it is obtained is destroyed or a non-`const` member function of the exception object is called.

30.4.3 Class `interrupt_token`

[`interrupt_token`]

- ¹ The class `interrupt_token` implements semantics of shared ownership of an interrupt state (an atomic token to signal an interrupt). An interrupt can only be signaled once. All owners can signal an interrupt, provided the token is valid. All owners can check whether an interrupt was signaled. The last remaining owner of the interrupt state is responsible for releasing the resources associated with the interrupt state.

```
namespace std {
    class interrupt_token {
    public:
        // 30.4.3.1 create, copy, destroy:
        explicit interrupt_token() noexcept;
        explicit interrupt_token(bool initial_state);

        interrupt_token(const interrupt_token&) noexcept;
        interrupt_token(interrupt_token&&) noexcept;
        interrupt_token& operator=(const interrupt_token&) noexcept;
        interrupt_token& operator=(interrupt_token&&) noexcept;
        ~interrupt_token();
    };
}
```

```

    void swap(interrupt_token&) noexcept;

    // 30.4.3.5 interrupt handling:
    bool valid() const noexcept;
    bool is_interrupted() const noexcept;
    bool interrupt();
    void throw_if_interrupted() const;
}

```

```

bool operator==(const interrupt_token& lhs, const interrupt_token& rhs);
bool operator!=(const interrupt_token& lhs, const interrupt_token& rhs);

```

Calls to `interrupt()`, `is_interrupted()`, and `throw_if_interrupted()` are atomic operations(6.8.2.1p3 ??) on an atomic object contained in the `interrupt_token`. Hence concurrent calls to these functions do not introduce data races. A call to `interrupt()` synchronizes with any call to `interrupt()`, `is_interrupted()`, or `throw_if_interrupted()` that observes the interrupt.

30.4.3.1 `interrupt_token` constructors [`interrupt__token.constr`]

```
interrupt_token() noexcept;
```

1 *Effects:* Constructs a new `interrupt_token` object that can't be used to signal interrupts. [Note: Therefore, no resources have to be associated for the state. — end note]

2 *Ensures:* `valid() == false`.

```
interrupt_token(bool initial_state) noexcept;
```

3 *Effects:* Constructs a new `interrupt_token` object that can signal interrupts via an atomic associated interrupt state.

4 *Ensures:* `valid() == true` and `is_interrupted() == initial_state`.

```
interrupt_token(const interrupt_token& rhs) noexcept;
```

5 *Effects:* If `rhs` is not valid, constructs an `interrupt_token` object that is not valid; otherwise, constructs an `interrupt_token` that shares the ownership of the interrupt state with `rhs`.

6 *Ensures:* `valid() == rhs.valid()` and `is_interrupted() == rhs.is_interrupted()` and `*this == rhs`.

```
interrupt_token(interrupt_token&& rhs) noexcept;
```

7 *Effects:* Move constructs an object of type `interrupt_token` from `rhs`.

8 *Ensures:* `*this` shall contain the old value of `rhs` and `rhs.valid() == false`.

30.4.3.2 `interrupt_token` destructor [`interrupt__token.destr`]

```
~interrupt_token();
```

1 *Effects:* If `valid()` and `*this` is the last owner of the interrupt state, releases the associated with the interrupt state.

30.4.3.3 `interrupt_token` assignment [`interrupt__token.assign`]

```
interrupt_token& operator=(const interrupt_token& rhs) noexcept;
```

1 *Effects:* Equivalent to: `interrupt_token(rhs).swap(*this);`

2 *Returns:* `*this`.

```
interrupt_token& operator=(interrupt_token&& rhs) noexcept;
```

3 *Effects:* Equivalent to: `interrupt_token(std::move(rhs)).swap(*this);`

4 *Returns:* `*this`.

30.4.3.4 `interrupt_token` swap [`interrupt__token.swap`]

```
void swap(interrupt_token& rhs) noexcept;
```

1 *Effects:* Swaps the state of `*this` and `rhs`.

30.4.3.5 interrupt_token members**[interrupt_token.mem]**`bool valid() const noexcept;`

1 *Returns:* `true` if the interrupt token can be used to signal interrupts.

`bool is_interrupted() const noexcept;`

2 *Returns:* `true` if initialized with `true` or initialized with `false` and `interrupt()` was called by one of the owners.

`bool interrupt();`

3 *Effects:* If `!valid()` or `is_interrupted()` the call has no effect. Otherwise, signals an interrupt so that `is_interrupted() == true`. [Note: Signaling an interrupt includes notifying all `condition_variables` temporarily registered via a an interruptable wait (30.6.1.2) — end note]

4 *Ensures:* `!valid() || is_interrupted()`

5 *Returns:* The value of `is_interrupted()` prior to the call.

`void throw_if_interrupted() const;`

6 *Effects:* Equivalent to:

```
    if (is_interrupted())
        throw interrupted();
```

30.4.3.6 interrupt_token comparisons**[interrupt_token.cmp]**`bool operator== (const interrupt_token& lhs, const interrupt_token& rhs);`

1 *Returns:* `!lhs.valid() && !rhs.valid()` or whether `lhs` and `rhs` refer to the same interrupt state (copied or moved from the same initial `interrupt_token` object).

`bool operator!= (const interrupt_token& lhs, const interrupt_token& rhs);`

2 *Returns:* `!(lhs==rhs)`.

30.5 Joining Threads

[thread.jthreads]

- ¹ 30.5 describes components that can be used to create and manage threads with the ability to signal interrupts to cooperatively cancel the running thread.

30.5.1 Header `<jthread>` synopsis

[thread.jthread.syn]

```
#include <interrupt_token>

namespace std {
    // 30.5.2 class jthread
    class jthread;

    void swap(jthread& x, jthread& y) noexcept;

    // 30.5.3 this_thread interrupt token handling
    namespace this_thread {
        static bool is_interrupted() noexcept;
        static void throw_if_interrupted();
        static interrupt_token get_interrupt_token() noexcept;
        static interrupt_token exchange_interrupt_token(const interrupt_token&) noexcept;
    }
}
```

[Editorial note: The `this_thread` extensions listed here might instead be added to "33.3.3 Namespace `this_thread` [thread.thread.this].]

30.5.2 Class `jthread`

[thread.jthread.class]

- ¹ The class `jthread` provides a mechanism to create a new thread of execution. The functionality is identical to class `thread` (??) with the additional ability to signal an interrupt and to automatically `join()` the started thread.

[Editorial note: This color signals differences to class `std::thread`.]

```
namespace std {
    class jthread {
    public:
        // types
        using id = thread::id;
        using native_handle_type = thread::native_handle_type;

        // construct/copy/destroy
        jthread() noexcept;
        template<class F, class... Args> explicit jthread(F&& f, Args&&... args);
        ~jthread();
        jthread(const jthread&) = delete;
        jthread(jthread&&) noexcept;
        jthread& operator=(const jthread&) = delete;
        jthread& operator=(jthread&&) noexcept;

        // members
        void swap(jthread&) noexcept;
        bool joinable() const noexcept;
        void join();
        void detach();
        id get_id() const noexcept;
        native_handle_type native_handle();    // see ??

        // interrupt token handling
        interrupt_token get_original_interrupt_token() const noexcept;
        bool interrupt() noexcept;

        // static members
        static unsigned int hardware_concurrency() noexcept;

    private:
        interrupt_token itoken;    // exposition only
    };
```



```

}

```

30.5.2.1 `jthread` constructors

[`thread.jthread.constr`]

```

jthread() noexcept;

```

1 *Effects:* Constructs a `jthread` object that does not represent a thread of execution.

2 *Ensures:* `get_id() == id()` and `itoken.valid() == false`.

```

template<class F, class... Args> explicit jthread(F&& f, Args&&... args);

```

3 *Requires:* `F` and each T_i in `Args` shall satisfy the *Cpp17MoveConstructible* requirements. *INVOKE*(*DECAY_COPY*(`std::forward<F>(f)`), *DECAY_COPY*(`std::forward<Args>(args)`)...) (??) shall be a valid expression.

4 *Remarks:* This constructor shall not participate in overload resolution if `remove_cvref_t<F>` is the same type as `std::jthread`.

5 *Effects:* Constructs an object of type `jthread`. The new thread of execution executes *INVOKE*(*DECAY_COPY*(`std::forward<F>(f)`), *DECAY_COPY*(`std::forward<Args>(args)`)...) with the calls to *DECAY_COPY* being evaluated in the constructing thread. Any return value from this invocation is ignored. [Note: This implies that any exceptions not thrown from the invocation of the copy of `f` will be thrown in the constructing thread, not the new thread. — end note] If the invocation of *INVOKE*(*DECAY_COPY*(`std::forward<F>(f)`), *DECAY_COPY*(`std::forward<Args>(args)`)...) terminates with an uncaught exception, `terminate` shall be called.

An uncaught `interrupted` exception in the started thread of execution will silently be ignored. [Note: Thus, an uncaught exception thrown by `this_thread::throw_if_interrupted()` will cause the started thread to end silently. — end note]

6 *Synchronization:* The completion of the invocation of the constructor synchronizes with the beginning of the invocation of the copy of `f`.

7 *Ensures:* `get_id() != id()`. `itoken.valid() == true`. `*this` represents the newly started thread. In the started thread of execution `this_thread::thread_itoken` is an `interrupt_token` equal to `itoken`. [Note: Note that the calling thread can signal an interrupt only once, because it can't replace this interrupt token. — end note]

8 *Throws:* `system_error` if unable to start the new thread.

9 *Error conditions:*

(9.1) — `resource_unavailable_try_again` — the system lacked the necessary resources to create another thread, or the system-imposed limit on the number of threads in a process would be exceeded.

```

jthread(jthread&& x) noexcept;

```

10 *Effects:* Constructs an object of type `jthread` from `x`, and sets `x` to a default constructed state.

11 *Ensures:* `x.get_id() == id()` and `get_id()` returns the value of `x.get_id()` prior to the start of construction. `itoken` yields the value of `x.itoken` prior to the start of construction and `x.itoken.valid() == false`.

30.5.2.2 `jthread` destructor

[`thread.jthread.destr`]

```

~jthread();

```

1 If `joinable()`, calls `interrupt()` and `join()`. Otherwise, has no effects. [Note: Operations on `*this` are not synchronized. — end note]

30.5.2.3 `jthread` assignment

[`thread.jthread.assign`]

```

jthread& operator=(jthread&& x) noexcept;

```

1 *Effects:* If `joinable()`, calls `interrupt()` and `join()`. Assigns the state of `x` to `*this` and sets `x` to a default constructed state.

2 *Ensures:* `x.get_id() == id()` and `get_id()` returns the value of `x.get_id()` prior to the assignment. `itoken` yields the value of `x.itoken` prior to the assignment and `x.itoken.valid() == false`.

3 *Returns:* `*this`.

30.5.2.4 `jthread` interrupt members**[`thread.jthread.interrupt`]**`interrupt_token get_original_interrupt_token() const noexcept`

1 *Effects:* Equivalent to: `return itoken;`

`bool interrupt() noexcept;`

2 *Effects:* Equivalent to: `return itoken.interrupt();`

30.5.3 Namespace `this_thread` Interrupt Handling**[`thread.jthread.this`]**

[*Editorial note:* The `this_thread` extensions listed here might instead be added to "33.3.3 Namespace `this_thread` [`thread.thread.this`].]

To be able to deal with signaled interrupt `this_thread` provides the following access to an `interrupt_token` (30.4.3):

```
namespace std::this_thread {
    interrupt_token thread_itoken;           // exposition only
    static interrupt_token get_interrupt_token() noexcept;
    static bool is_interrupted() noexcept;
    static void throw_if_interrupted();
    static interrupt_token exchange_interrupt_token(const interrupt_token&) noexcept;
}
```

For any thread of execution, `thread_itoken` is default initialized unless the thread was started with a constructor of class `jthread` (30.5.2).

`interrupt_token get_interrupt_token() noexcept;`

1 *Returns:* `this_thread::thread_itoken`.

`bool is_interrupted() noexcept;`

2 *Returns:* `this_thread::get_interrupt_token().is_interrupted()`.

`void throw_if_interrupted();`

3 *Effects:* Equivalent to: `this_thread::get_interrupt_token().throw_if_interrupted();`

`interrupt_token exchange_interrupt_token(const interrupt_token& it) noexcept;`

4 *Effects:* Equivalent to: `this_thread::itoken = it;`

5 *Returns:* `this_thread::thread_itoken` prior to the exchange.

[*Note:* By calling this function for the first time on an object `this_thread::get_interrupt_token()` will no longer signal interrupts from the calling thread unless the returned token is restored. — *end note*]

30.6 Condition variables

[thread.condition]

30.6.1 Class condition_variable

[thread.condition.condvar]

```

namespace std {
    class condition_variable {
    public:
        condition_variable();
        ~condition_variable();

        condition_variable(const condition_variable&) = delete;
        condition_variable& operator=(const condition_variable&) = delete;

        void notify_one() noexcept;
        void notify_all() noexcept;

        // 30.6.1.1 noninterruptable waits:
        void wait(unique_lock<mutex>& lock);
        template<class Predicate>
            void wait(unique_lock<mutex>& lock, Predicate pred);

        template<class Clock, class Duration>
            cv_status wait_until(unique_lock<mutex>& lock,
                               const chrono::time_point<Clock, Duration>& abs_time);
        template<class Clock, class Duration, class Predicate>
            bool wait_until(unique_lock<mutex>& lock,
                           const chrono::time_point<Clock, Duration>& abs_time,
                           Predicate pred);

        template<class Rep, class Period>
            cv_status wait_for(unique_lock<mutex>& lock,
                              const chrono::duration<Rep, Period>& rel_time);
        template<class Rep, class Period, class Predicate>
            bool wait_for(unique_lock<mutex>& lock,
                          const chrono::duration<Rep, Period>& rel_time,
                          Predicate pred);

        // 30.6.1.2 interruptable waits:
        template<class Predicate>
            void iwait(unique_lock<mutex>& lock, Predicate pred);

        template<class Clock, class Duration, class Predicate>
            bool iwait_until(unique_lock<mutex>& lock,
                             const chrono::time_point<Clock, Duration>& abs_time,
                             Predicate pred);
        template<class Rep, class Period, class Predicate>
            bool iwait_for(unique_lock<mutex>& lock,
                           const chrono::duration<Rep, Period>& rel_time,
                           Predicate pred);

        // 30.6.1.3 interrupt_token waits:
        template <class Predicate>
            bool wait_until(unique_lock<mutex>& lock,
                           Predicate pred,
                           interrupt_token itoken);
        template <class Clock, class Duration, class Predicate>
            bool wait_until(unique_lock<mutex>& lock,
                           const chrono::time_point<Clock, Duration>& abs_time,
                           Predicate pred,
                           interrupt_token itoken);
        template <class Rep, class Period, class Predicate>
            bool wait_for(unique_lock<mutex>& lock,
                          const chrono::duration<Rep, Period>& rel_time,
                          Predicate pred,
                          interrupt_token itoken);
    };
}

```

```

        using native_handle_type = implementation-defined;           // see ??
        native_handle_type native_handle();                          // see ??
    };
}

```

1 The class `condition_variable` shall be a standard-layout class (??).

```
condition_variable();
```

2 *Effects:* Constructs an object of type `condition_variable`.

3 *Throws:* `system_error` when an exception is required (??).

4 *Error conditions:*

(4.1) — `resource_unavailable_try_again` — if some non-memory resource limitation prevents initialization.

```
~condition_variable();
```

5 *Requires:* There shall be no thread blocked on `*this`. [Note: That is, all threads shall have been notified; they may subsequently block on the lock specified in the wait. This relaxes the usual rules, which would have required all wait calls to happen before destruction. Only the notification to unblock the wait needs to happen before destruction. The user should take care to ensure that no threads wait on `*this` once the destructor has been started, especially when the waiting threads are calling the wait functions in a loop or using the overloads of `wait`, `wait_for`, or `wait_until` that take a predicate. — end note]

6 *Effects:* Destroys the object.

```
void notify_one() noexcept;
```

7 *Effects:* If any threads are blocked waiting for `*this`, unblocks one of those threads.

```
void notify_all() noexcept;
```

30.6.1.1 Noninterruptable waits

[`thread.condition.wait`]

1 *Effects:* Unblocks all threads that are blocked waiting for `*this`.

```
void wait(unique_lock<mutex>& lock);
```

2 *Requires:* `lock.owns_lock()` is true and `lock.mutex()` is locked by the calling thread, and either

(2.1) — no other thread is waiting on this `condition_variable` object or

(2.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, `wait_until`, `await`, `await_for`, or `await_until`) threads.

3 *Effects:*

(3.1) — Atomically calls `lock.unlock()` and blocks on `*this`.

(3.2) — When unblocked, calls `lock.lock()` (possibly blocking on the lock), then returns.

(3.3) — The function will unblock when signaled by a call to `notify_one()` or a call to `notify_all()`, or spuriously.

4 *Remarks:* If the function fails to meet the postcondition, `terminate()` shall be called (??). [Note: This can happen if the re-locking of the mutex throws an exception. — end note]

5 *Ensures:* `lock.owns_lock()` is true and `lock.mutex()` is locked by the calling thread.

6 *Throws:* Nothing.

```
template<class Predicate>
```

```
void wait(unique_lock<mutex>& lock, Predicate pred);
```

7 *Requires:* `lock.owns_lock()` is true and `lock.mutex()` is locked by the calling thread, and either

(7.1) — no other thread is waiting on this `condition_variable` object or

(7.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, `wait_until`, `await`, `await_for`, or `await_until`) threads.

8 *Effects:* Equivalent to:

```
while (!pred())
    wait(lock);
```

9 *Remarks:* If the function fails to meet the postcondition, `terminate()` shall be called (??). [*Note:* This can happen if the re-locking of the mutex throws an exception. — *end note*]

10 *Ensures:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

11 *Throws:* Any exception thrown by `pred`.

```
template<class Clock, class Duration>
cv_status wait_until(unique_lock<mutex>& lock,
                    const chrono::time_point<Clock, Duration>& abs_time);
```

12 *Requires:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either

(12.1) — no other thread is waiting on this `condition_variable` object or

(12.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, `wait_until`, `await`, `await_for`, or `await_until`) threads.

13 *Effects:*

(13.1) — Atomically calls `lock.unlock()` and blocks on `*this`.

(13.2) — When unblocked, calls `lock.lock()` (possibly blocking on the lock), then returns.

(13.3) — The function will unblock when signaled by a call to `notify_one()`, a call to `notify_all()`, expiration of the absolute timeout (??) specified by `abs_time`, or spuriously.

(13.4) — If the function exits via an exception, `lock.lock()` shall be called prior to exiting the function.

14 *Remarks:* If the function fails to meet the postcondition, `terminate()` shall be called (??). [*Note:* This can happen if the re-locking of the mutex throws an exception. — *end note*]

15 *Ensures:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

16 *Returns:* `cv_status::timeout` if the absolute timeout (??) specified by `abs_time` expired, otherwise `cv_status::no_timeout`.

17 *Throws:* Timeout-related exceptions (??).

```
template<class Rep, class Period>
cv_status wait_for(unique_lock<mutex>& lock,
                  const chrono::duration<Rep, Period>& rel_time);
```

18 *Requires:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either

(18.1) — no other thread is waiting on this `condition_variable` object or

(18.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, `wait_until`, `await`, `await_for`, or `await_until`) threads.

19 *Effects:* Equivalent to:

```
return wait_until(lock, chrono::steady_clock::now() + rel_time);
```

20 *Returns:* `cv_status::timeout` if the relative timeout (??) specified by `rel_time` expired, otherwise `cv_status::no_timeout`.

21 *Remarks:* If the function fails to meet the postcondition, `terminate()` shall be called (??). [*Note:* This can happen if the re-locking of the mutex throws an exception. — *end note*]

22 *Ensures:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

23 *Throws:* Timeout-related exceptions (??).

```
template<class Clock, class Duration, class Predicate>
bool wait_until(unique_lock<mutex>& lock,
               const chrono::time_point<Clock, Duration>& abs_time,
               Predicate pred);
```

24 *Requires:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either

(24.1) — no other thread is waiting on this `condition_variable` object or

(24.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, `wait_until`, `await`, `await_for`, or `await_until`) threads.

25 *Effects:* Equivalent to:

```
while (!pred())
    if (wait_until(lock, abs_time) == cv_status::timeout)
        return pred();
return true;
```

26 *Remarks:* If the function fails to meet the postcondition, `terminate()` shall be called (??). [Note: This can happen if the re-locking of the mutex throws an exception. — end note]

27 *Ensures:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

28 [Note: The returned value indicates whether the predicate evaluated to `true` regardless of whether the timeout was triggered. — end note]

29 *Throws:* Timeout-related exceptions (??) or any exception thrown by `pred`.

```
template<class Rep, class Period, class Predicate>
bool wait_for(unique_lock<mutex>& lock,
              const chrono::duration<Rep, Period>& rel_time,
              Predicate pred);
```

30 *Requires:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either

(30.1) — no other thread is waiting on this `condition_variable` object or

(30.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, `wait_until`, `await`, `await_for`, or `await_until`) threads.

31 *Effects:* Equivalent to:

```
return wait_until(lock, chrono::steady_clock::now() + rel_time, std::move(pred));
```

32 [Note: There is no blocking if `pred()` is initially `true`, even if the timeout has already expired. — end note]

33 *Remarks:* If the function fails to meet the postcondition, `terminate()` shall be called (??). [Note: This can happen if the re-locking of the mutex throws an exception. — end note]

34 *Ensures:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

35 [Note: The returned value indicates whether the predicate evaluates to `true` regardless of whether the timeout was triggered. — end note]

36 *Throws:* Timeout-related exceptions (??) or any exception thrown by `pred`.

30.6.1.2 Interruptable waits

[`thread.condition.await`]

The following functions ensure to get notified if an interrupt is signaled for `this_thread::get_interrupt_token()`. In that case they throw `interrupted` as if `this_thread::throw_if_interrupted()` was called.

[Editorial note: Because all signatures here in the effects clause call `is_interrupted()` or `throw_if_interrupted()`, we don't need wording that the calls synchronize with `interrupt()`.]

[Editorial note: This color signals differences to the corresponding `wait...` functions.]

```
template<class Predicate>
void await(unique_lock<mutex>& lock, Predicate pred);
```

1 *Requires:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either

(1.1) — no other thread is waiting on this `condition_variable` object or

(1.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, `wait_until`, `await`, `await_for`, or `await_until`) threads.

2 *Effects:* Registers `*this` to get notified when an interrupt is signaled on `this_thread::get_interrupt_token()` during this call and then equivalent to:

```
while(!pred()) {
    this_thread::throw_if_interrupted();
    cv.wait(lock, [&pred] {
        return pred() || this_thread::is_interrupted();
    });
}
```

```
    });
}
```

3 *Remarks:* If the function fails to meet the postcondition, `terminate()` shall be called (??). [*Note:* This can happen if the re-locking of the mutex throws an exception. — *end note*]

4 *Ensures:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

5 *Throws:* Any exception thrown by `pred` or exception `interrupted` if `this_thread::is_interrupted()`.

```
template<class Clock, class Duration, class Predicate>
bool iwait_until(unique_lock<mutex>& lock,
                const chrono::time_point<Clock, Duration>& abs_time,
                Predicate pred);
```

6 *Requires:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either
 (6.1) — no other thread is waiting on this `condition_variable` object or
 (6.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, `wait_until`, `await`, `await_for`, or `await_until`) threads.

7 *Effects:* Registers `*this` to get notified when an interrupt is signaled on `this_thread::get_interrupt_token()` during this call and then equivalent to:

```
while(!pred() && Clock::now() < abs_time) {
    this_thread::throw_if_interrupted();
    cv.wait_until(lock, abs_time,
                 [&pred] {
                     return pred() || this_thread::is_interrupted();
                 });
}
return pred();
```

8 *Remarks:* If the function fails to meet the postcondition, `terminate()` shall be called (??). [*Note:* This can happen if the re-locking of the mutex throws an exception. — *end note*]

9 *Ensures:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

10 [*Note:* The returned value indicates whether the predicate evaluated to `true` regardless of whether the timeout was triggered or an interrupt was signaled. — *end note*]

11 *Throws:* Timeout-related exceptions (??) or any exception thrown by `pred` or exception `interrupted` if `this_thread::is_interrupted()`.

```
template<class Rep, class Period, class Predicate>
bool iwait_for(unique_lock<mutex>& lock,
               const chrono::duration<Rep, Period>& rel_time,
               Predicate pred);
```

12 *Requires:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either
 (12.1) — no other thread is waiting on this `condition_variable` object or
 (12.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, `wait_until`, `await`, `await_for`, or `await_until`) threads.

13 *Effects:* Equivalent to:

```
this_thread::throw_if_interrupted();
return iwait_until(lock,
                  std::chrono::steady_clock::now() + rel_time,
                  std::move(pred));
```

14 *Remarks:* If the function fails to meet the postcondition, `terminate()` shall be called (??). [*Note:* This can happen if the re-locking of the mutex throws an exception. — *end note*]

15 *Ensures:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

16 [*Note:* The returned value indicates whether the predicate evaluated to `true` regardless of whether the timeout was triggered or an interrupt was signaled. — *end note*]

17 *Throws:* Timeout-related exceptions (??) or any exception thrown by `pred` or exception `interrupted` if `this_thread::is_interrupted()`.

30.6.1.3 interrupt_token waits[`thread.condition.interrupt_token`]

The following functions ensure to get notified if an interrupt is signaled for the passed `interrupt_token`. In that case they return `false` (if the predicate evaluates to `false`).

[*Editorial note:* Because all signatures here in the effects clause call `is_interrupted()` or `throw_if_interrupted()`, we don't need wording that the calls synchronize with `interrupt()`.]

```
template <class Predicate>
    bool wait_until(unique_lock<mutex>& lock,
                   Predicate pred,
                   interrupt_token itoken);
```

[*Editorial note:* This color signals differences to the corresponding `wait()` function without the interrupt token parameter.]

- 1 *Requires:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either
 - (1.1) — no other thread is waiting on this `condition_variable` object or
 - (1.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, `wait_until`, `await`, `await_for`, or `await_until`) threads.
- 2 *Effects:* Registers `*this` to get notified when an interrupt is signaled on `itoken` during this call and then equivalent to:


```
while(!pred() && !itoken.is_interrupted()) {
    cv.wait(lock, [&pred, &itoken] {
        return pred() || itoken.is_interrupted();
    });
}
return pred();
```
- 3 [*Note:* The returned value indicates whether the predicate evaluated to `true` regardless of whether the timeout was triggered or an interrupt was signaled. — *end note*]
- 4 *Ensures:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.
- 5 *Remarks:* If the function fails to meet the postcondition, `terminate()` shall be called (??). [*Note:* This can happen if the re-locking of the mutex throws an exception. — *end note*]
- 6 *Throws:* Any exception thrown by `pred`.

```
template <class Clock, class Duration, class Predicate>
    bool wait_until(unique_lock<mutex>& lock,
                   const chrono::time_point<Clock, Duration>& abs_time
                   Predicate pred,
                   interrupt_token itoken);
```

[*Editorial note:* This color signals differences to the corresponding `wait_until()` function without the interrupt token parameter.]

- 7 *Requires:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either
 - (7.1) — no other thread is waiting on this `condition_variable` object or
 - (7.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, `wait_until`, `await`, `await_for`, or `await_until`) threads.
- 8 *Effects:* Registers `*this` to get notified when an interrupt is signaled on `itoken` during this call and then equivalent to:


```
while(!pred() && !itoken.is_interrupted() && Clock::now() < abs_time) {
    cv.wait_until(lock,
                  abs_time,
                  [&pred, &itoken] {
                      return pred() || itoken.is_interrupted();
                  });
}
return pred();
```
- 9 [*Note:* The returned value indicates whether the predicate evaluated to `true` regardless of whether the timeout was triggered or an interrupt was signaled. — *end note*]

10 *Ensures:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

11 *Remarks:* If the function fails to meet the postcondition, `terminate()` shall be called (??). [*Note:* This can happen if the re-locking of the mutex throws an exception. — *end note*]

12 *Throws:* Timeout-related exceptions (??) or any exception thrown by `pred`.

```
template <class Rep, class Period, class Predicate>
    bool wait_for(unique_lock<mutex>& lock,
                  const chrono::duration<Rep, Period>& rel_time,
                  Predicate pred,
                  interrupt_token itoken);
```

[*Editorial note:* This color signals differences to the corresponding `wait_for()` function without the interrupt token parameter.]

13 *Requires:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either

(13.1) — no other thread is waiting on this `condition_variable` object or

(13.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, `wait_until`, `await`, `await_for`, or `await_until`) threads.

14 *Effects:* Equivalent to:

```
    return wait_until(lock, chrono::steady_clock::now() + rel_time, std::move(pred),
                     std::move(itoken));
```

15 [*Note:* The returned value indicates whether the predicate evaluates to `true` regardless of whether the timeout was triggered or an interrupt was signaled. — *end note*]

16 [*Note:* There is no blocking if `pred()` is initially `true`, even if the timeout has already expired. — *end note*]

17 *Ensures:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

18 *Remarks:* If the function fails to meet the postcondition, `terminate()` shall be called (??). [*Note:* This can happen if the re-locking of the mutex throws an exception. — *end note*]

19 *Throws:* Timeout-related exceptions (??) or any exception thrown by `pred`.